SMART ATTENDANCE PRO

College Project Report | Python, OpenCV, Tkinter and SQLite

An on-device, camera-based attendance demonstration with local admin login, student enrollment, face classification, date-filtered attendance reports, and CSV export.

1. Abstract

This project demonstrates how a local Python desktop application can register consenting student face samples, train a Local Binary Patterns Histogram (LBPH) classifier, and record attendance upon repeated matching. SQLite maintains student, attendance and administrator credential records. A Tkinter dashboard shows daily metrics and a seven-day chart; reports can be filtered by date and exported to CSV. The application is intended as a supervised educational prototype, not an unsupervised biometric identity system.

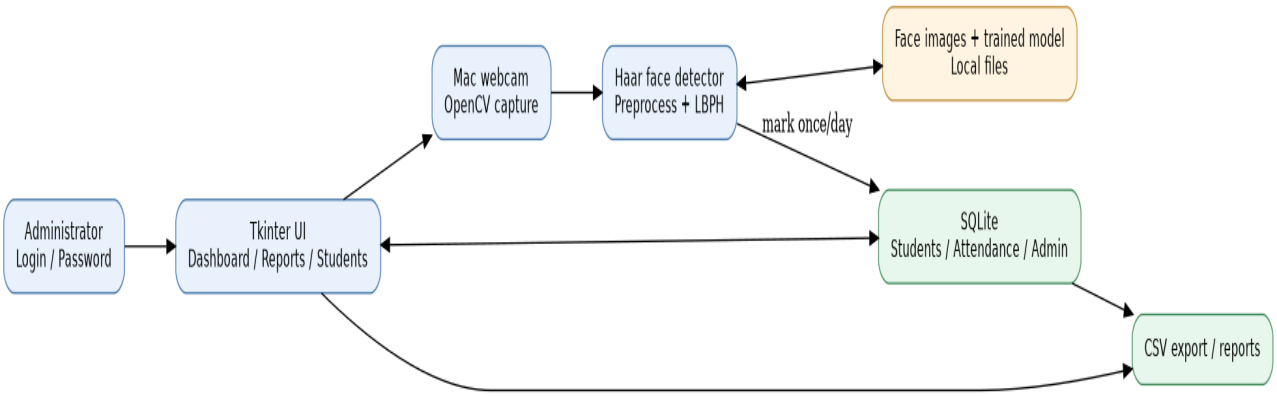
2. Problem statement and objectives

Manual paper attendance requires repetitive entry and can introduce transcription errors. The project aims to support student registration, repeated-frame face matching, one daily attendance record per student, local role-restricted access, and convenient CSV reporting. It does not guarantee protection from attendance fraud or camera spoofing.

3. Tools and technology

Layer	Implementation
Language	Python 3.14 environment (tested syntactically)
Interface	Tkinter and Pillow image preview
Computer vision	OpenCV Haar cascade + LBPH
Storage	SQLite, on-device face sample images
Exports	Python csv module (Excel/Numbers compatible)
Security	PBKDF2-HMAC-SHA256 password hashes + salted credentials

4. Architecture



5. Main modules

Authentication: Admin account initialization, login, logout, and password changes. **Enrollment:** Capture 25 face samples and retrain the model. **Recognition:** Detect faces, normalize crops, evaluate LBPH distance, require multiple consecutive matches, and record attendance. **Dashboard:** Current student counts, attendance rate and seven-day chart. **Reports:** Date-filtered rows, monthly shortcut, check-in totals and CSV export.

6. Database design

students(student_id PRIMARY KEY, name, created_at); **attendance**(id PRIMARY KEY, student_id FOREIGN KEY, attendance_date, attendance_time, UNIQUE(student_id, attendance_date)); **admin_auth**(username PRIMARY KEY, salt, hash, iterations). The unique attendance constraint blocks duplicate entries for the same calendar date.

7. Recognition procedure

The webcam image is converted to grayscale. Haar cascade detection locates candidate faces. Each face is resized to 200 x 200 and histogram-equalized. An LBPH recognizer predicts the closest enrolled label and returns a distance score. A distance below a configurable threshold, sustained across five consecutive frames, triggers an INSERT OR IGNORE attendance write. LBPH distance is not a statistical confidence percentage. Threshold calibration and independent test images are required for meaningful accuracy claims.

8. Testing plan

Verify: first-run admin creation; correct/incorrect passwords; enrollment and 25 saved face crops; model creation; correct identification under multiple lighting conditions; rejection of unregistered faces; repeated detections producing one daily attendance row; date filtering, month shortcut, and CSV contents; restart persistence; camera permission handling; full student data deletion.

9. Privacy, ethical and technical limitations

Use only with informed consent and secure local storage. Do not upload face images or model files to public repositories. This prototype has no liveness detection and may be fooled by photos, videos, or visually similar faces. Face recognition can have demographic and environmental accuracy differences. Use human review and a non-biometric fallback. “Not marked today” is not proof of absence when no timetable and attendance session rules are defined.

10. Future enhancements

Optional human confirmation of uncertain matches; secure session-level attendance and timetable mapping; evaluation on a held-out dataset; calibrated thresholds and accuracy metrics; stronger access control, encrypted local storage, retention/deletion rules, and explicit liveness or supervised check-in options.

11. Conclusion

Smart Attendance Pro illustrates an end-to-end Python application that combines user interface design, computer vision, data persistence, reporting, and password-based administrative access. It is suitable for a college demonstration once webcam access and recognition reliability have been locally validated.